

DESIGN AND IMPLEMENTATION OF A SELF-DIAGNOSING ASYNCHRONOUS FIFO WITH RUNTIME CDC FAULT DETECTION

A COURSE BASED PROJECT REPORT

submitted by

V SAI CHARAN GOUD 1602-23-735-316

M LAXMANA CHARY 1602-23-735-318

to
the partial fulfillment of the academic requirements for the award of the degree
of
Bachelor of Engineering
in
Electronics & Communication Engineering



Department of Electronics & Communication Engineering

VASAVI COLLEGE OF ENGINEERING (AUTONOMOUS)

IBRAHIMBAGH, HYDERABAD-500031

May 2026

DECLARATION

We, the undersigned, declare that the project report **DESIGN AND IMPLEMENTATION OF A SELF-DIAGNOSING ASYNCHRONOUS FIFO WITH RUNTIME CDC FAULT DETECTION** submitted for partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering of Vasavi College of Engineering (Autonomous), Ibrahimbagh, Hyderabad is a bonafide work done by me under supervision of Dr.P.SATHISH and Dr.K.DEEPTHI This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the sources. We also declare we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources that have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma, or similar title of any other University.

Signature of student :

Name of student : V SAI CHARAN GOUD

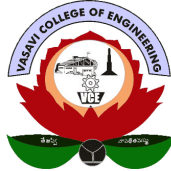
Signature of student :

Name of student : M LAXMANA CHARY

Place :

Date : May 11, 2026

DEPARTMENT OF ECE
VASAVI COLLEGE OF ENGINEERING (AUTONOMOUS)
ACCREDITED BY NAAC WITH 'A+' GRADE
IBRAHIMBAGH, HYDERABAD-500031



CERTIFICATE

This is to certify that the report entitled **DESIGN AND IMPLEMENTATION OF A SELF-DIAGNOSING ASYNCHRONOUS FIFO WITH RUNTIME CDC FAULT DETECTION** submitted by

V SAI CHARAN GOUD 1602-23-735-316
M LAXMANA CHARY 1602-23-735-318

students of Electronics and Communication Engineering Department, Vasavi College of Engineering in partial fulfilment of the requirement for the award of the degree of Bachelor of Engineering in Electronics & Communication Engineering is a record of the bonafide work carried out by them during the academic year 2025-2026. The results embodied in this project report has not been submitted to any other university or institute for the award of any degree.

Supervisors

Dr.P. SATHISH
Associate Professor – Dept of ECE

Dr.K.DEEPTHI
Associate Professor – Dept of ECE

Head of the Department
Dr.E. SREENIVASA RAO
Professor and HOD
Vasavi College of Engineering (Autonomous)

Signature of The External Examiner

ACKNOWLEDGMENT

I wish to record my indebtedness and thankfulness to all who helped me prepare this Project Report titled **DESIGN AND IMPLEMENTATION OF A SELF-DIAGNOSING ASYNCHRONOUS FIFO WITH RUNTIME CDC FAULT DETECTION** and present it satisfactorily.

I am especially grateful to my guide and supervisor Dr.P.SATHISH & Dr.K.DEEPTHI in the Department of Electronics & Communication Engineering for giving me valuable suggestions and critical inputs in the preparation of this report. I am also thankful to Prof.E.SREENIVASA RAO, Head of Department of Electronics & Communication Engineering for encouragement.

My friends in my class have always been helpful and I am grateful to them for patiently listening to my presentations on my work related to the Project.

V SAI CHARAN GOUD

M LAXMANA CHARY

B.E. (Electronics & Communication Engineering)

Department of Electronics & Communication Engineering

VASAVI COLLEGE OF ENGINEERING (AUTONOMOUS)

ABSTRACT

Modern digital systems frequently involve communication across multiple clock domains, making reliable Clock Domain Crossing (CDC) a critical challenge in VLSI design. Asynchronous FIFOs are widely used to enable safe data transfer between independent clock domains; however, conventional implementations primarily focus on synchronization and lack mechanisms for detecting runtime faults arising from metastability and pointer inconsistencies.

This project presents the design and implementation of a **self-diagnosing asynchronous FIFO with real-time CDC fault detection**, developed using Verilog HDL. The architecture utilizes Gray-coded pointer synchronization and multi-stage flip-flop synchronizers to minimize metastability risks during cross-domain communication. In addition, a dedicated diagnostic unit is integrated to detect critical anomalies such as Gray code violations, overflow and underflow conditions, and pointer stall behavior during runtime.

Unlike traditional FIFO designs, the proposed system enhances reliability and observability by continuously monitoring internal CDC behavior and generating fault flags without affecting normal operation. The design is fully parameterized, synthesizable, and verified using a comprehensive testbench with asynchronous clock scenarios and boundary condition testing.

Simulation results validate correct FIFO operation across independent clock domains and demonstrate effective detection of fault conditions. The proposed design is well-suited for integration into modern System-on-Chip (SoC) architectures and high-speed interfaces, contributing toward improved robustness in CDC-critical digital systems.

CONTENTS

ACKNOWLEDGMENT	i
ABSTRACT	ii
LIST OF TABLES	iv
LIST OF FIGURES	v
1 Introduction	1
1.1 Basic Overview	1
1.2 Research Motivation	2
1.3 Problem Statement	3
1.4 Objectives of the Work	4
1.5 Organization of the Report	4
2 Literature Survey	6
3 Design of Self-Diagnosing Asynchronous FIFO Architecture	9
3.1 System Overview	9
3.2 Design Methodology	9
3.3 System Architecture	10
3.4 Write Domain Design	11
3.5 Read Domain Design	12
3.6 Pointer Synchronization	12
3.7 FIFO Memory Design	13
3.8 Diagnostic Unit Design	13
3.9 Design Features	14

4	RTL Implementation and Functional Verification of Asynchronous FIFO	15
4.1	Implementation Strategy	15
4.2	RTL Design Considerations	16
4.3	Module Integration	16
4.4	Diagnostic Logic Implementation	17
4.5	Simulation Setup	17
4.6	Verification Approach	18
5	Results and Discussion	19
5.1	Functional Verification of FIFO Operation	19
5.2	Waveform Analysis	20
5.3	Overflow and Underflow Detection	21
5.4	Pointer Synchronization and CDC Analysis	22
5.5	Diagnostic Unit Evaluation	23
5.6	Fault Injection and Diagnostic Validation	24
5.7	Performance Observation	26
5.8	Practical Applications of the Proposed FIFO	27
5.9	Summary of Results	28
6	Conclusion and Future Scope	29
6.1	Conclusion	29
6.2	Future Scope	30
	MAPPING TO UN SUSTAINABLE DEVELOPMENT GOALS (SDGs)	31
	REFERENCES	32
	APPENDIX	33
	Appendix A: Design Code	33
	Appendix B: Testbench	52

LIST OF TABLES

2.1	Comparison of Existing Asynchronous FIFO Designs and Proposed Work	7
-----	--	---

LIST OF FIGURES

1.1	Block Diagram of Self-Diagnosing Asynchronous FIFO	1
3.1	Overall Architecture of the Asynchronous FIFO	10
4.1	Block Diagram of Asynchronous FIFO	15
5.1	Asynchronous FIFO Architecture	19
5.2	Simulation Waveform of Asynchronous FIFO Operation	20
5.3	Fault Injection and Diagnostic Validation Waveform	25

Chapter 1

INTRODUCTION

1.1 BASIC OVERVIEW

Modern System-on-Chip (SoC) designs increasingly rely on multiple clock domains to meet performance, power, and modularity requirements. Communication between these independent clock domains introduces significant challenges due to differences in clock frequency, phase, and timing uncertainty. This issue, commonly referred to as Clock Domain Crossing (CDC), can lead to metastability, data corruption, and unpredictable system behavior if not handled properly.

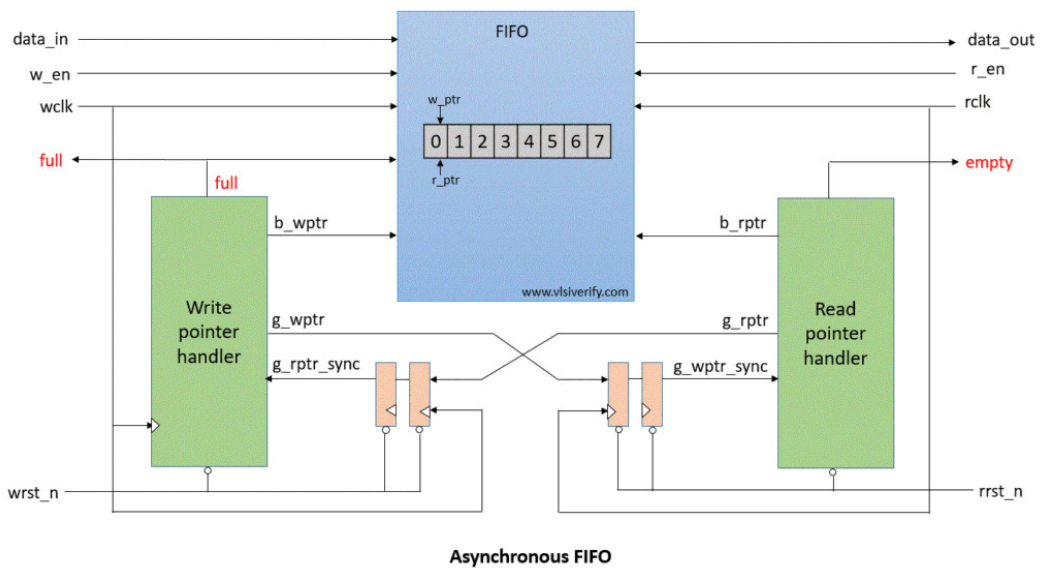


Figure 1.1: Block Diagram of Self-Diagnosing Asynchronous FIFO

Asynchronous First-In-First-Out (FIFO) buffers are widely used as a standard solution for safe data transfer across clock domains. These FIFOs operate

with separate read and write clocks, allowing independent operation of producer and consumer modules. Reliable operation is achieved using techniques such as Gray-coded pointer synchronization and multi-stage flip-flop synchronizers to minimize metastability during pointer transfer.

However, conventional FIFO designs primarily focus on ensuring correct data transfer and synchronization, with limited emphasis on runtime reliability monitoring. In practical VLSI systems, undetected CDC-related failures can propagate silently and lead to critical system-level issues.

This project presents a **self-diagnosing asynchronous FIFO with runtime CDC fault detection**, which not only ensures safe data transfer but also actively monitors internal FIFO behavior. The design incorporates diagnostic mechanisms to detect anomalies such as Gray code violations, overflow and underflow conditions, and pointer stall scenarios, thereby improving system robustness and observability.

1.2 RESEARCH MOTIVATION

In modern digital systems, especially high-speed communication interfaces and processor subsystems, reliable data transfer across clock domains is a fundamental requirement. While traditional asynchronous FIFO designs provide effective synchronization, they lack built-in mechanisms to detect runtime faults associated with CDC behavior.

In real-world applications, issues such as invalid pointer transitions, overflow and underflow conditions, and pointer stalling may arise due to design bugs, timing violations, or unexpected operating conditions. These faults are often difficult to detect during simulation and may only appear intermittently during runtime.

Designing a self-diagnosing FIFO architecture enables continuous monitor-

ing of internal behavior and early detection of CDC-related anomalies. This enhances system reliability, simplifies debugging, and aligns with industry practices in designing fault-aware and robust digital systems.

This project is motivated by the need to bridge the gap between conventional FIFO designs and reliability-driven architectures required in modern VLSI systems.

1.3 PROBLEM STATEMENT

Conventional asynchronous FIFO designs ensure safe data transfer across clock domains using synchronization techniques but do not provide mechanisms to detect runtime faults associated with CDC behavior. As a result, issues such as metastability-induced errors, pointer inconsistencies, overflow and underflow conditions, and stalled pointer behavior may go undetected during operation.

The lack of built-in fault detection reduces system observability and increases the complexity of debugging in large-scale designs. Therefore, there is a need for an enhanced FIFO architecture that not only guarantees correct data transfer but also incorporates real-time fault detection mechanisms.

Therefore, the problem addressed in this project is:

“To design and implement a self-diagnosing asynchronous FIFO in Verilog that ensures reliable data transfer across independent clock domains while detecting and reporting runtime CDC-related faults such as Gray code violations, overflow, underflow, and pointer stall conditions.”

1.4 OBJECTIVES OF THE WORK

- To design a dual-clock asynchronous FIFO supporting independent read and write domains.
- To implement Gray-coded pointer synchronization for safe clock domain crossing.
- To utilize multi-stage synchronizers to reduce metastability risks.
- To develop a diagnostic unit capable of detecting Gray code violations and pointer inconsistencies.
- To detect overflow and underflow conditions with sticky fault indicators.
- To identify pointer stall conditions using runtime monitoring logic.
- To verify the design using a comprehensive testbench with asynchronous clock scenarios.
- To ensure the design is parameterized, synthesizable, and suitable for FPGA/ASIC implementation.

1.5 ORGANIZATION OF THE REPORT

- **Chapter 1 - Introduction**

Presents the basic overview, motivation, problem statement, and objectives of the proposed work.

- **Chapter 2 - Literature Survey**

Reviews existing techniques related to asynchronous FIFO design and clock domain crossing mechanisms.

- **Chapter 3 - Design Methodology and Implementation**

Describes the system architecture, pointer synchronization techniques, FIFO memory design, and diagnostic logic.

- **Chapter 4 - Implementation and Simulation**

Details the Verilog implementation, module-level functionality, and simulation environment.

- **Chapter 5 - Results and Discussion**

Analyzes functional correctness, fault detection capability, and performance behavior of the design.

- **Chapter 6 - Conclusion and Future Scope**

Summarizes the work and discusses possible enhancements for advanced CDC handling and system integration.

Chapter 2

LITERATURE SURVEY

Clock Domain Crossing (CDC) has been a critical area of research in modern VLSI systems due to the increasing use of multiple clock domains in System-on-Chip (SoC) architectures. Improper handling of CDC can lead to metastability, data corruption, and unpredictable system behavior, making reliable synchronization techniques essential [1, 6].

Asynchronous First-In-First-Out (FIFO) buffers are widely recognized as an effective solution for safe data transfer between independent clock domains. Traditional FIFO designs employ Gray-coded pointers and multi-stage synchronizers to minimize metastability during pointer transfer [3, 8]. The work presented in [3] demonstrates a robust asynchronous FIFO design with emphasis on functional correctness and verification, while the classical approach by Cummings [7] establishes industry-standard techniques for FIFO implementation using Gray code pointer synchronization.

Further research has focused on improving performance and efficiency of asynchronous FIFOs. The architecture proposed in [2] introduces memory banking and clock-enable techniques to enhance energy efficiency, highlighting the importance of optimizing FIFO structures for modern high-speed systems. Similarly, formal verification approaches presented in [4] ensure correctness of FIFO operation under various CDC conditions, emphasizing the need for rigorous validation methodologies.

Despite these advancements, most existing designs primarily focus on data integrity, synchronization correctness, and performance optimization. Limited at-

tention has been given to runtime fault detection and observability of internal FIFO behavior. In practical systems, faults such as pointer inconsistencies, Gray code violations, overflow, underflow, and pointer stalling may occur due to metastability effects or unexpected operating conditions. These issues are often difficult to detect using conventional verification techniques alone.

Existing literature lacks integrated mechanisms for real-time monitoring and detection of CDC-related anomalies during operation. This gap highlights the need for enhanced FIFO architectures that incorporate diagnostic capabilities to improve reliability and debug visibility.

To address this limitation, the present work proposes a **self-diagnosing asynchronous FIFO with runtime CDC fault detection**. Unlike conventional designs, the proposed architecture integrates a dedicated diagnostic unit capable of detecting Gray code violations, overflow and underflow conditions, and pointer stall behavior during runtime. This approach improves system robustness and aligns with modern VLSI design practices that emphasize reliability, fault awareness, and observability.

Table 2.1: Comparison of Existing Asynchronous FIFO Designs and Proposed Work

Year & Author	Title of Work	Key Contribution / Limitation
2025 Bhoomika K. M. et al. [1]	Timing Analysis of CDC in Asynchronous FIFO	Focuses on timing issues and CDC behavior; lacks runtime fault detection mechanisms.
2025 Dinghan Chen et al. [2]	Energy-Efficient Asynchronous FIFO Architecture	Improves power efficiency using memory banking; does not address fault monitoring.

Continued on next page

Table 2.1: Comparison of Existing Asynchronous FIFO Designs and Proposed Work
(Continued)

Year & Author	Title of Work	Key Contribution / Limitation
2021 Mittal et al. [3]	Design and Verification of Asynchronous FIFO	Emphasizes functional correctness and verification; no diagnostic capability included.
2020 Yu et al. [4]	Formal Verification of Asynchronous FIFO Designs	Ensures design correctness through formal methods; lacks runtime observability.
2002 Cliff Cummings [7]	Simulation and Synthesis Techniques for FIFO	Establishes Gray code pointer synchronization methodology; no fault detection features.
2022 Dobkin et al. [8]	High-Speed FIFO using Gray Code Pointers	Optimizes speed and pointer design; does not include reliability monitoring.
Proposed Work	Self-Diagnosing Asynchronous FIFO	Integrates CDC-safe design with real-time fault detection (Gray violations, overflow/underflow, pointer stall), improving reliability and observability.

Chapter 3

DESIGN OF SELF-DIAGNOSING ASYNCHRONOUS FIFO ARCHITECTURE

3.1 SYSTEM OVERVIEW

The proposed system is a **self-diagnosing asynchronous FIFO** designed to enable reliable data transfer between two independent clock domains. The architecture consists of separate write and read domains operating on independent clocks, along with synchronization and diagnostic units to ensure safe and observable operation.

The FIFO design is parameterized in terms of data width and depth, making it scalable and suitable for integration into various System-on-Chip (SoC) applications. The system incorporates Gray-coded pointer synchronization and multi-stage synchronizers to mitigate metastability issues during clock domain crossing (CDC).

In addition to standard FIFO functionality, a dedicated diagnostic module is integrated to monitor runtime behavior and detect anomalies such as Gray code violations, overflow, underflow, and pointer stall conditions.

3.2 DESIGN METHODOLOGY

The design follows a modular RTL-based approach using Verilog HDL. Each functional block of the FIFO is implemented as an independent module, enabling better readability, scalability, and verification.

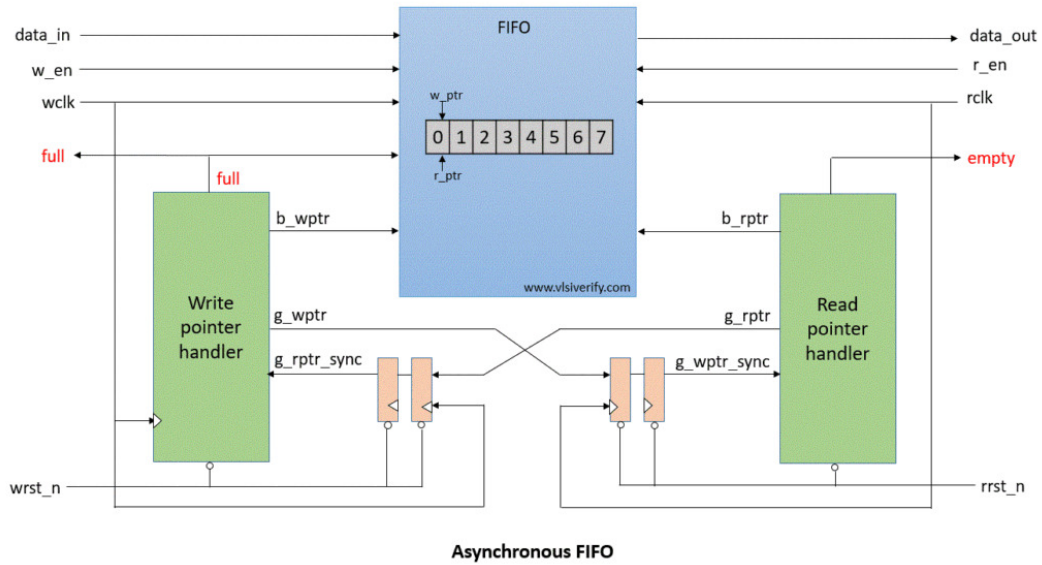


Figure 3.1: Overall Architecture of the Asynchronous FIFO

The overall design flow includes:

- Partitioning the FIFO into write domain, read domain, memory, synchronization, and diagnostic units
- Implementing Gray-coded pointers for safe CDC transfer
- Using multi-stage synchronizers to reduce metastability probability
- Designing control logic for full and empty detection
- Integrating diagnostic logic for runtime fault monitoring
- Verifying functionality using a testbench with asynchronous clock inputs

3.3 SYSTEM ARCHITECTURE

The asynchronous FIFO architecture consists of the following major components:

- Write Domain (Write Pointer and Control Logic)
- Read Domain (Read Pointer and Control Logic)
- Pointer Synchronization Units
- FIFO Memory Block
- Diagnostic Unit

Each component operates either in the write clock domain or read clock domain, ensuring proper isolation and synchronization across domains.

3.4 WRITE DOMAIN DESIGN

The write domain operates on the write clock (`wr_clk`) and is responsible for accepting input data and storing it into the FIFO memory.

Key functionalities include:

- Binary write pointer generation
- Conversion of binary pointer to Gray code
- Full condition detection using synchronized read pointer
- Controlled write enable logic to prevent overflow

The write pointer is incremented only when write enable is asserted and the FIFO is not full. The full condition is detected using a comparison between the next Gray-coded write pointer and the synchronized read pointer.

Overflow conditions are also detected and flagged when a write operation is attempted while the FIFO is full.

3.5 READ DOMAIN DESIGN

The read domain operates on the read clock (`rd_clk`) and is responsible for retrieving data from the FIFO.

Key functionalities include:

- Binary read pointer generation
- Conversion to Gray-coded pointer
- Empty condition detection using synchronized write pointer
- Valid data indication and underflow detection

The read pointer is incremented only when read enable is asserted and the FIFO is not empty. The empty condition is detected when the Gray-coded read pointer matches the synchronized write pointer.

Underflow conditions are flagged when a read operation is attempted while the FIFO is empty.

3.6 POINTER SYNCHRONIZATION

Pointer synchronization is a critical aspect of asynchronous FIFO design. Direct transfer of binary pointers across clock domains may lead to metastability issues. To address this, Gray-coded pointers are used, ensuring that only one bit changes between consecutive values.

Multi-stage flip-flop synchronizers are employed to safely transfer Gray-coded pointers between clock domains. This significantly reduces the probability of metastability affecting system operation.

Separate synchronizers are used for:

- Read pointer synchronization into the write domain

- Write pointer synchronization into the read domain

3.7 FIFO MEMORY DESIGN

The FIFO memory is implemented as a dual-port memory structure, allowing simultaneous read and write operations using independent clocks.

Key features include:

- Parameterized memory depth and data width
- Independent read and write addressing
- Synchronous write and read operations

The memory block ensures efficient data storage and retrieval without causing contention between read and write operations.

3.8 DIAGNOSTIC UNIT DESIGN

A key contribution of this work is the integration of a diagnostic unit for runtime fault detection.

The diagnostic module continuously monitors FIFO operation and detects the following anomalies:

- **Gray Code Violation:** Detects invalid transitions in Gray-coded pointers
- **Overflow Condition:** Detects write attempts when FIFO is full
- **Underflow Condition:** Detects read attempts when FIFO is empty
- **Pointer Stall Detection:** Identifies cases where pointers fail to update despite valid operations

Sticky fault flags are used to retain error conditions once detected, improving observability and aiding debugging.

3.9 DESIGN FEATURES

The proposed FIFO design offers the following key features:

- Robust CDC handling using Gray code and synchronizers
- Parameterized and scalable architecture
- Runtime fault detection capability
- Sticky fault indicators for debugging
- Fully synthesizable RTL design
- Suitable for FPGA and ASIC implementation

Chapter 4

RTL IMPLEMENTATION AND FUNCTIONAL VERIFICATION OF ASYNCHRONOUS FIFO

4.1 IMPLEMENTATION STRATEGY

The asynchronous FIFO is implemented using a modular RTL design approach in Verilog HDL. The design is structured to clearly separate clock domains and ensure safe communication between them.

A key design decision is the use of parameterization for data width, FIFO depth, and synchronization stages. This allows the design to be easily scaled and reused across different applications without modifying the core logic.

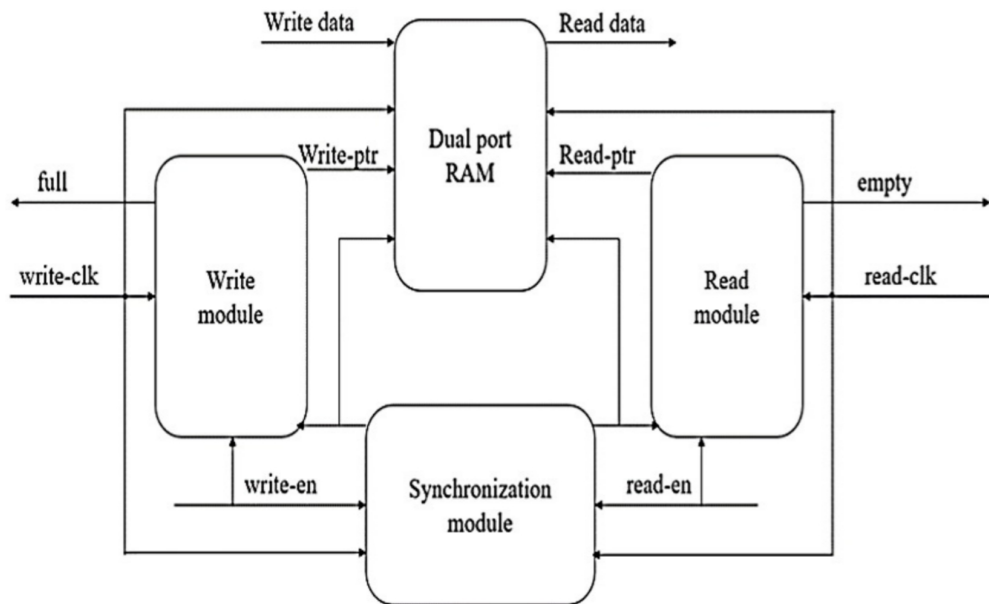


Figure 4.1: Block Diagram of Asynchronous FIFO

Special attention is given to clock domain isolation, ensuring that no combinational paths exist between write and read domains.

4.2 RTL DESIGN CONSIDERATIONS

Several important RTL-level considerations are incorporated during implementation:

- **Clock Domain Isolation:** All signals crossing domains are strictly synchronized to avoid metastability.
- **Gray Code Usage:** Binary pointers are converted to Gray code before crossing clock domains to ensure only single-bit transitions.
- **Glitch-Free Flags:** Full and empty flags are generated using registered logic to prevent glitches.
- **Safe Enable Logic:** Write and read operations are conditionally allowed to avoid overflow and underflow.

These decisions ensure that the FIFO behaves reliably under asynchronous conditions.

4.3 MODULE INTEGRATION

The top-level module integrates all submodules while maintaining strict separation between clock domains.

Key integration aspects include:

- Controlled routing of synchronized pointers between domains

- Proper interfacing between memory and control logic
- Clean separation of write and read paths

The modular structure improves readability and simplifies debugging.

4.4 DIAGNOSTIC LOGIC IMPLEMENTATION

The diagnostic unit is implemented as an independent monitoring block operating alongside the main FIFO logic.

Unlike traditional designs, this module does not interfere with data flow but continuously observes internal signals.

Key implementation insights:

- Bitwise comparison logic is used to detect invalid Gray transitions
- Sticky flags are implemented using register latching
- Counter-based logic is used for detecting pointer stall conditions

This approach ensures non-intrusive fault monitoring.

4.5 SIMULATION SETUP

The design is verified using a custom testbench that generates independent clock signals for read and write domains.

The write and read clocks operate at different frequencies to emulate real asynchronous conditions. Reset signals are applied independently to both domains.

Test stimulus includes sequential writes, controlled reads, and intentional boundary condition testing.

4.6 VERIFICATION APPROACH

Instead of only checking outputs, the verification strategy focuses on observing internal behavior.

Key checks include:

- Correct pointer progression across clock domains
- Proper assertion of full and empty flags
- Detection of overflow and underflow conditions
- Validation of diagnostic outputs under fault scenarios

This approach ensures both functional correctness and robustness of the design.

Chapter 5

RESULTS AND DISCUSSION

5.1 FUNCTIONAL VERIFICATION OF FIFO OPERATION

The functionality of the proposed asynchronous FIFO is verified using simulation with independent read and write clocks. The system is tested under normal operating conditions to ensure correct data transfer between clock domains.

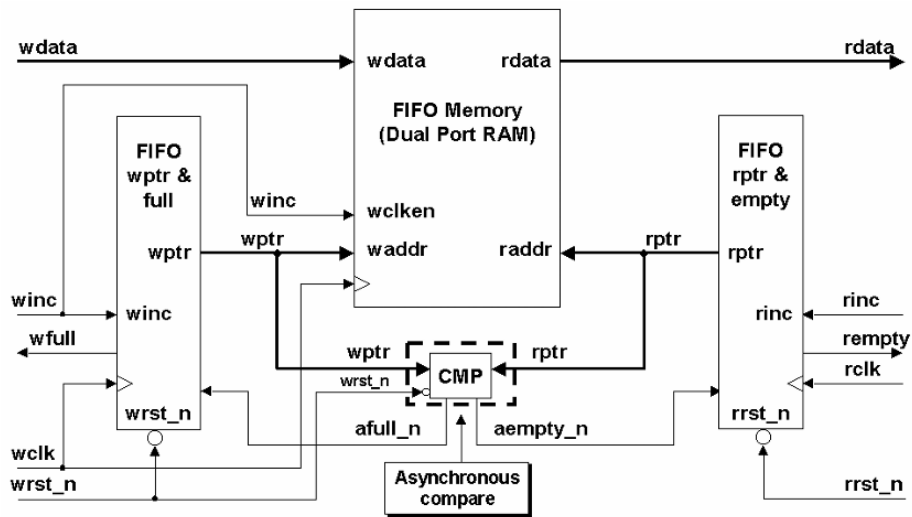


Figure 5.1: Asynchronous FIFO Architecture

The write and read domains operate independently using different clock frequencies, thereby emulating realistic asynchronous conditions commonly found in modern System-on-Chip (SoC) architectures. Data written into the FIFO from the write domain is successfully retrieved in the same order from the read domain, confirming correct FIFO functionality.

The simulation validates reliable communication between independent clock

domains without introducing data corruption, synchronization failure, or ordering mismatch. The design successfully maintains stable FIFO operation even under asynchronous timing conditions.

The correct operation of the FIFO confirms the effectiveness of Gray-coded pointer synchronization and multi-stage synchronization logic implemented in the architecture.

5.2 WAVEFORM ANALYSIS

The waveform in Fig. 5.2 illustrates the detailed behavior of key FIFO signals during asynchronous operation.

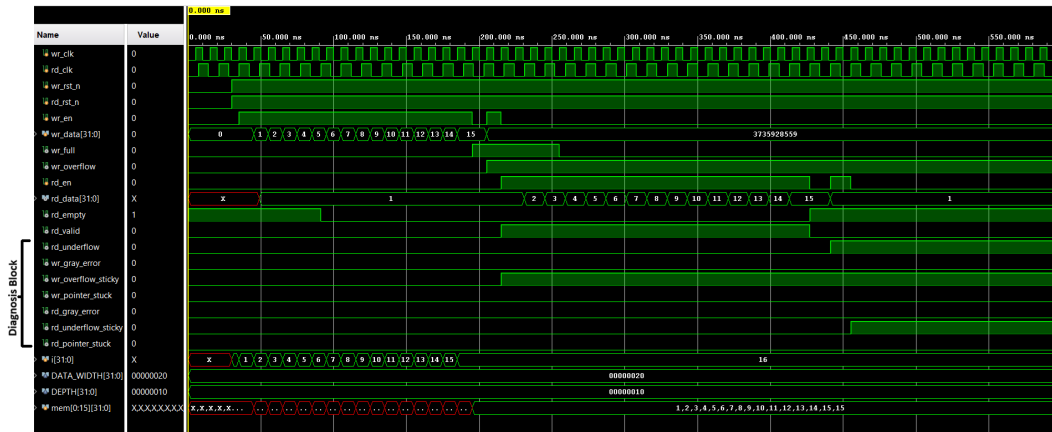


Figure 5.2: Simulation Waveform of Asynchronous FIFO Operation

During the write phase, sequential data values are applied at the input and stored into FIFO memory under the control of the write clock (wr_clk). The waveform confirms that valid write operations occur only when the FIFO is not full.

The write pointer increments correctly during every successful write operation, and the corresponding Gray-coded pointer transitions remain stable throughout simulation. The synchronized read pointer received from the opposite clock domain also updates correctly without generating invalid transitions.

During the read phase, stored data is retrieved sequentially using the read clock (`rd_clk`). The output data sequence matches the input sequence, validating correct FIFO ordering behavior. The waveform also confirms proper assertion of the `rd_valid` signal whenever valid data becomes available at the output.

The asynchronous nature of the design is clearly visible from the difference in read and write clock frequencies. Despite operating under different clock domains, the FIFO maintains reliable synchronization and stable operation.

The waveform additionally demonstrates correct generation of status signals such as `wr_full`, `rd_empty`, and `rd_valid`. These signals accurately reflect the internal FIFO state during operation and confirm correct implementation of pointer comparison logic.

The simulation also verifies that memory operations occur without collision or corruption, demonstrating correct coordination between memory access logic and synchronization circuitry.

5.3 OVERFLOW AND UNDERFLOW DETECTION

Reliable handling of boundary conditions is critical in asynchronous FIFO architectures. The proposed design includes dedicated mechanisms for detecting overflow and underflow conditions during runtime.

Overflow occurs when a write operation is attempted while the FIFO is already full. During simulation, continuous write operations are performed until the FIFO reaches maximum capacity. Once the FIFO becomes full, the `wr_full` signal is asserted, preventing additional write operations from modifying memory contents.

When an intentional write request is applied after the FIFO becomes full, the `wr_overflow` signal is asserted successfully. The corresponding sticky flag

(`wr_overflow_sticky`) remains active even after the transient condition disappears, thereby improving fault visibility.

Similarly, underflow occurs when a read operation is attempted while the FIFO is empty. After all stored data is retrieved from the FIFO, the `rd_empty` signal becomes active. Any further read request under this condition triggers the `rd_underflow` signal.

The `rd_underflow_sticky` signal successfully latches the underflow event, ensuring that the fault condition remains observable during debugging and runtime analysis.

The implementation of sticky fault indicators significantly improves system observability and simplifies failure analysis in complex digital systems.

The successful detection of overflow and underflow conditions confirms the reliability of the FIFO control logic and validates safe boundary operation under asynchronous conditions.

5.4 POINTER SYNCHRONIZATION AND CDC ANALYSIS

Clock Domain Crossing (CDC) reliability is one of the most important aspects of asynchronous FIFO design. Since the write and read domains operate independently, improper synchronization can lead to metastability, pointer corruption, or incorrect status generation.

The proposed architecture mitigates these issues using Gray-coded pointer synchronization combined with multi-stage flip-flop synchronizers.

Binary pointers are converted into Gray code before crossing clock domains. Since only one bit changes between consecutive Gray-code values, the probability of incorrect multi-bit sampling is significantly reduced.

Simulation results confirm stable and correct synchronization of both read

and write pointers across asynchronous domains. No invalid pointer transitions or synchronization glitches are observed during normal operation.

The synchronized pointers update correctly even under different read and write clock frequencies, demonstrating the effectiveness of the implemented CDC strategy.

The synchronization stages provide sufficient settling time before signals are sampled in the destination domain, thereby reducing the possibility of metastability propagation.

The waveform confirms that:

- Pointer synchronization remains stable throughout operation
- No metastability-induced behavior is observed
- FIFO status signals are generated correctly
- Data integrity is maintained during asynchronous communication

The implemented synchronization methodology significantly improves the robustness and reliability of the FIFO architecture for practical VLSI applications.

5.5 DIAGNOSTIC UNIT EVALUATION

A major contribution of the proposed work is the integration of a dedicated diagnostic unit capable of monitoring FIFO behavior during runtime.

Unlike conventional asynchronous FIFO designs that focus only on synchronization, the proposed architecture continuously observes internal FIFO activity and reports abnormal conditions using dedicated diagnostic outputs.

Simulation results confirm the following functionalities:

- **Gray Code Monitoring:** Gray-code transitions are continuously checked to identify invalid multi-bit transitions during pointer synchronization.

- **Overflow and Underflow Monitoring:** Overflow and underflow conditions are detected and retained using sticky fault indicators.
- **Pointer Stability Monitoring:** Pointer stall detection logic continuously checks whether pointers update correctly during valid operations.
- **Runtime Observation:** Internal FIFO activity is monitored without affecting normal data transfer functionality.
- **Fault Visibility:** Dedicated fault outputs improve debugging capability and runtime observability.
- **Non-Intrusive Monitoring:** The diagnostic module operates independently without introducing additional delay in the FIFO data path.

The diagnostic outputs provide valuable runtime insight into internal FIFO behavior and improve the reliability of the overall architecture.

The integrated diagnostic capability is particularly useful in reliability-critical systems where early fault detection and runtime monitoring are essential.

5.6 FAULT INJECTION AND DIAGNOSTIC VALIDATION

To validate the effectiveness of the proposed diagnostic architecture, fault conditions were intentionally injected during simulation using Vivado force techniques. Artificial Gray-code violations and pointer stall conditions were introduced to verify runtime fault detection capability under abnormal operating scenarios.

The waveform in Fig. 5.3 confirms successful detection of multiple fault conditions by the diagnostic unit.

Invalid Gray-code transitions were intentionally introduced by forcing abnormal pointer transitions during simulation. Under normal Gray-code operation,

only a single bit changes between consecutive pointer values. However, during fault injection, multiple bits were intentionally toggled simultaneously to emulate synchronization abnormalities and CDC-related failures.

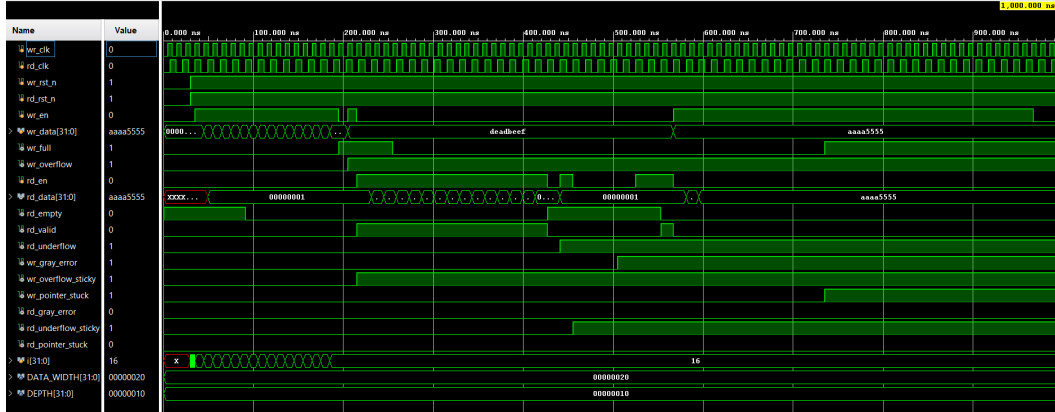


Figure 5.3: Fault Injection and Diagnostic Validation Waveform

The diagnostic module successfully detected the invalid multi-bit transition and asserted the `wr_gray_error` signal. This confirms the correctness of the implemented Gray-code monitoring logic and demonstrates its ability to identify synchronization inconsistencies during runtime.

Similarly, pointer stall conditions were artificially created by freezing the write pointer while maintaining active write operations. Since the pointer remained unchanged for multiple clock cycles despite continuous write enable activity, the diagnostic module interpreted this condition as abnormal pointer behavior.

After the configured threshold duration was exceeded, the `wr_pointer_stuck` signal was asserted successfully. This validates the effectiveness of the implemented pointer stability monitoring mechanism.

The waveform additionally confirms successful assertion of overflow and underflow sticky fault indicators during invalid FIFO operations. Once asserted, the sticky fault flags retained their status throughout the remaining simulation period, thereby improving runtime observability and simplifying debugging analysis.

The fault injection analysis demonstrates that the proposed architecture is

capable of detecting both transient and persistent fault conditions without interrupting normal FIFO functionality. Unlike conventional asynchronous FIFOs that focus only on safe data transfer, the proposed design actively monitors internal FIFO behavior and provides dedicated diagnostic visibility.

The successful detection of intentionally injected faults validates the runtime monitoring capability of the self-diagnosing FIFO architecture and confirms the reliability of the integrated diagnostic framework.

The simulation results also demonstrate that the diagnostic mechanisms operate independently from the FIFO data path. Therefore, fault monitoring is achieved without introducing additional delay or affecting normal data transfer performance.

The ability to intentionally inject and detect faults highlights the verification-oriented nature of the design methodology adopted in this work. Such runtime fault-awareness mechanisms are highly beneficial in modern VLSI systems where reliability, observability, and fault tolerance are critical design requirements.

Overall, the fault injection experiments confirm that the proposed asynchronous FIFO architecture not only achieves reliable Clock Domain Crossing communication but also provides enhanced runtime reliability monitoring and diagnostic capability for CDC-critical digital systems.

5.7 PERFORMANCE OBSERVATION

The asynchronous FIFO demonstrates stable and efficient operation under different write and read clock frequencies.

The implemented synchronization logic introduces minimal latency while maintaining reliable CDC operation. The FIFO continuously transfers data without unnecessary stalls during normal operation.

Key observations from simulation include:

- Continuous and ordered data transfer between asynchronous domains
- Correct assertion of full and empty flags
- Reliable overflow and underflow detection
- Stable Gray-code synchronization behavior
- Successful runtime diagnostic monitoring

The parameterized architecture also enables scalability for different FIFO depths and data widths without requiring major modifications to the RTL structure.

The modular implementation approach improves reusability and simplifies future enhancements such as protocol integration or additional fault detection mechanisms.

Overall, the design achieves a balance between reliability, scalability, and efficient asynchronous communication.

5.8 PRACTICAL APPLICATIONS OF THE PROPOSED FIFO

The proposed asynchronous FIFO architecture can be integrated into a wide range of modern digital systems where reliable Clock Domain Crossing (CDC) communication is required.

Potential application areas include:

- System-on-Chip (SoC) communication infrastructure
- UART and serial communication interfaces
- AXI and APB-based communication systems
- Embedded processors and microcontroller subsystems
- High-speed buffering and streaming applications

- Multi-clock digital communication architectures

The integrated diagnostic and runtime monitoring features make the proposed architecture particularly suitable for reliability-critical applications requiring enhanced fault observability.

The design methodology adopted in this work can also be extended to advanced fault-aware communication systems and next-generation VLSI architectures.

5.9 SUMMARY OF RESULTS

The simulation and verification results confirm that the proposed self-diagnosing asynchronous FIFO successfully achieves reliable Clock Domain Crossing communication under asynchronous operating conditions.

The architecture ensures correct FIFO behavior, stable pointer synchronization, and safe boundary operation using Gray-coded pointers and multi-stage synchronizers.

The integrated diagnostic unit successfully detects overflow, underflow, Gray-code violations, and pointer stall conditions during runtime without affecting normal data transfer operation.

The fault injection analysis further validates the effectiveness of the diagnostic mechanisms and demonstrates the robustness of the proposed architecture.

Overall, the design demonstrates a reliable, scalable, and fault-aware asynchronous FIFO architecture suitable for modern VLSI systems and CDC-critical applications.

Chapter 6

CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

This project presents the design and implementation of a **self-diagnosing asynchronous FIFO** with runtime Clock Domain Crossing (CDC) fault detection using Verilog HDL. The design successfully enables reliable data transfer between independent clock domains using Gray-coded pointer synchronization and multi-stage synchronizers to mitigate metastability issues.

Unlike conventional FIFO implementations, the proposed architecture incorporates a dedicated diagnostic unit capable of detecting critical runtime anomalies such as Gray code violations, overflow and underflow conditions, and pointer stall behavior. These features enhance system observability and improve reliability in CDC-critical environments.

The design is fully parameterized, modular, and synthesizable, making it suitable for integration into modern System-on-Chip (SoC) architectures. Simulation results validate correct FIFO functionality, accurate boundary condition handling, and effective fault detection without impacting normal operation.

Overall, this work demonstrates a practical and reliability-oriented approach to asynchronous FIFO design, aligning with industry requirements for robust and fault-aware digital systems.

6.2 FUTURE SCOPE

The proposed design can be further enhanced in several ways to meet advanced VLSI system requirements:

- **Formal Verification:** Integration of formal verification techniques to mathematically prove correctness of CDC behavior.
- **Advanced CDC Handling:** Extension to support multi-bit data synchronization schemes and additional CDC-safe protocols.
- **Error Recovery Mechanisms:** Incorporation of automatic recovery logic for detected faults instead of only flagging them.
- **Power Optimization:** Implementation of low-power techniques such as clock gating and memory optimization.
- **FPGA/ASIC Validation:** Deployment of the design on FPGA platforms and further synthesis for ASIC-level validation.
- **Integration with SoC Interfaces:** Extension of the FIFO for use in communication protocols such as UART, AXI, or APB interfaces.

MAPPING TO UN SUSTAINABLE DEVELOPMENT GOALS (SDGs)

The design and implementation of the **Self-Diagnosing Asynchronous FIFO with Run-time Clock Domain Crossing (CDC) Fault Detection** aligns with the Sustainable Development Goals (SDGs) established by the United Nations. The project contributes toward reliable digital infrastructure, advanced VLSI system design, and technological innovation in modern System-on-Chip (SoC) architectures.

Primary Goal: SDG 9 — Industry, Innovation and Infrastructure

Target 9.5: Enhance scientific research and upgrade technological capabilities of industrial sectors.

This project contributes to reliable digital communication infrastructure through the implementation of a CDC-safe asynchronous FIFO architecture using Verilog HDL. The design addresses multi-clock synchronization challenges using Gray-coded pointer synchronization, metastability-aware techniques, and runtime fault detection mechanisms.

The integration of diagnostic features such as Gray code violation detection, overflow and underflow monitoring, and pointer stall detection improves the reliability and observability of modern VLSI systems used in communication and embedded applications.

Secondary Goal: SDG 4 — Quality Education

Target 4.4: Increase the number of people with relevant technical and engineering skills for employment and innovation.

This project contributes to engineering education by providing practical exposure to asynchronous FIFO design, CDC techniques, RTL design, and hardware verification methodologies. The implementation using Verilog HDL and simulation tools helps students gain hands-on experience in modern VLSI system development.

Overall, this project supports sustainable technological development and engineering education by promoting reliable digital infrastructure and practical learning in advanced hardware system design.

REFERENCES

- [1] Bhoomika K. M., Gokul Krishnan V., Peketi Reshma, Sree Vidya J., and Shylashree N., “Timing Analysis of Clock Domain Crossing in Asynchronous FIFO,” *IEEE*, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/11211483>
- [2] Dinghan Chen and Rongze Fan, “Energy-Efficient Asynchronous FIFO Architecture with Memory Banking and Clock-Enable Output Register,” *IEEE*, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/11307762>
- [3] S. Mittal and A. Gupta, “Design and Verification of Asynchronous FIFO for Robust Clock Domain Crossing,” *IEEE Access*, vol. 9, pp. 102345–102356, 2021.
- [4] J. Yu and P. Mishra, “Formal Verification of Asynchronous FIFO Designs for CDC Reliability,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 12, pp. 2560–2572, 2020.
- [5] D. J. Kinniment, *Synchronization and Arbitration in Digital Systems*, Springer, 2020.
- [6] M. Greenstreet, J. Sparsø, and S. Furber, “Metastability-Aware Design for Reliable Clock Domain Crossing,” *IEEE*, 2021.
- [7] Cliff Cummings, “Simulation and Synthesis Techniques for Asynchronous FIFO Design,” *SNUG Conference*, 2002.
- [8] R. Dobkin, R. Ginosar, and C. P. Sotiriou, “High-Speed Asynchronous FIFO Design Using Gray Code Pointers,” *IEEE*, 2022.

APPENDIX A: DESIGN CODE

Note: Key RTL modules are included below. Complete source code is available upon request.

```
// ASYNCHRONOUS FIFO TOP MODULE
```

```
module async_fifo_top #(
parameter DATA_WIDTH = 32,
          DEPTH = 16,
          SYNC_STAGES = 2 )(

// Write Side Interface
input wr_clk,
input wr_rst_n,
input wr_en,
input [DATA_WIDTH-1 : 0]wr_data,
output wr_full,
output wr_overflow,

// Read Side Interface
input rd_clk,
input rd_rst_n,
input rd_en,
output [DATA_WIDTH-1 : 0]rd_data,
output rd_empty,
output rd_valid,
output rd_underflow,
```

```

    // Write diagnostic outputs
    output wr_gray_error,
    output wr_overflow_sticky,
    output wr_pointer_stuck,

    // Read diagnostic outputs
    output rd_gray_error,
    output rd_underflow_sticky,
    output rd_pointer_stuck
);

localparam ADDR_WIDTH = $clog2(DEPTH);
localparam PTR_WIDTH  = ADDR_WIDTH + 1;

// Between Write Domain and Sync
wire [PTR_WIDTH-1:0] wr_gray_ptr;
wire [PTR_WIDTH-1:0] rd_gray_ptr;
// Synchronized Versions
wire [PTR_WIDTH-1:0] wr_gray_sync;
wire [PTR_WIDTH-1:0] rd_gray_sync;
// Memory Interface
wire [ADDR_WIDTH-1:0] wr_addr;
wire [ADDR_WIDTH-1:0] rd_addr;
wire wr_allow;

```

```

wr_domain #(
    .DATA_WIDTH(DATA_WIDTH),
    .DEPTH(DEPTH),
    .ADDR_WIDTH(ADDR_WIDTH),
    .PTR_WIDTH(PTR_WIDTH),
    .SYNC_STAGES(SYNC_STAGES)
) WR_Domain(
    .wr_clk(wr_clk),
    .wr_rst_n(wr_rst_n),
    .wr_en(wr_en),
    .rd_gray_sync(rd_gray_sync),
    .wr_gray_ptr(wr_gray_ptr),
    .wr_addr(wr_addr),
    .wr_allow(wr_allow),
    .wr_full(wr_full),
    .wr_overflow(wr_overflow)
);

```

```

rd_domain #(
    .DATA_WIDTH(DATA_WIDTH),
    .DEPTH(DEPTH),
    .ADDR_WIDTH(ADDR_WIDTH),
    .PTR_WIDTH(PTR_WIDTH),
    .SYNC_STAGES(SYNC_STAGES)
) RD_Domain (

```

```

        .rd_clk(rd_clk),
        .rd_rst_n(rd_rst_n),
        .rd_en(rd_en),
        .wr_gray_sync(wr_gray_sync),
        .rd_gray_ptr(rd_gray_ptr),
        .rd_addr(rd_addr),
        .rd_valid(rd_valid),
        .rd_empty(rd_empty),
        .rd_underflow(rd_underflow)
    );

// Read pointer ? Write domain
sync_ptr #(
    .PTR_WIDTH(PTR_WIDTH),
    .SYNC_STAGES(SYNC_STAGES)
) u_sync_rd2wr (
    .clk_dest(wr_clk),
    .rst_dest_n(wr_rst_n),
    .ptr_gray_src(rd_gray_ptr),
    .ptr_gray_sync(rd_gray_sync)
);

// Write pointer ? Read domain
sync_ptr #(
    .PTR_WIDTH(PTR_WIDTH),
    .SYNC_STAGES(SYNC_STAGES)
) u_sync_wr2rd (

```

```

        .clk_dest(rd_clk),
        .rst_dest_n(rd_rst_n),
        .ptr_gray_src(wr_gray_ptr),
        .ptr_gray_sync(wr_gray_sync)
    );

```

```

// Instantiate Memory

```

```

fifo_mem #(
    .DATA_WIDTH(DATA_WIDTH),
    .DEPTH(DEPTH),
    .ADDR_WIDTH(ADDR_WIDTH)
) u_mem (
    .wr_clk(wr_clk),
    .wr_en(wr_allow),
    .wr_addr(wr_addr),
    .wr_data(wr_data),

    .rd_clk(rd_clk),
    .rd_addr(rd_addr),
    .rd_data(rd_data)
);

```

```

fifo_diag #(
    .PTR_WIDTH(PTR_WIDTH),
    .STUCK_THRESHOLD(16)
) u_fifo_diag (

```

```

// Write domain
.wr_clk(wr_clk),
.wr_rst_n(wr_rst_n),
.wr_en(wr_en),
.wr_full(wr_full),
.wr_gray_ptr(wr_gray_ptr),
.wr_overflow(wr_overflow),

// Read domain
.rd_clk(rd_clk),
.rd_rst_n(rd_rst_n),
.rd_en(rd_en),
.rd_empty(rd_empty),
.rd_gray_ptr(rd_gray_ptr),
.rd_underflow(rd_underflow),

// Write outputs
.wr_gray_error(wr_gray_error),
.wr_overflow_sticky(wr_overflow_sticky),
.wr_pointer_stuck(wr_pointer_stuck),

// Read outputs
.rd_gray_error(rd_gray_error),
.rd_underflow_sticky(rd_underflow_sticky),
.rd_pointer_stuck(rd_pointer_stuck)
);

```

```

initial begin
    if ((DEPTH & (DEPTH-1)) != 0)
        $error("DEPTH must be power of 2");
    end

endmodule

// WRITE DOMAIN

module wr_domain #(
    parameter DATA_WIDTH = 32,
        DEPTH = 16,
        ADDR_WIDTH = $clog2(DEPTH),
        PTR_WIDTH = ADDR_WIDTH + 1,
        SYNC_STAGES = 2 )(

    input wr_clk,
    input wr_rst_n,
    input wr_en,
    input [PTR_WIDTH-1 : 0]rd_gray_sync,

    output [PTR_WIDTH-1 : 0]wr_gray_ptr,
    output [ADDR_WIDTH-1 : 0]wr_addr,
    output wr_allow,
    output wr_full,
    output wr_overflow

```

```

    );

    reg [PTR_WIDTH-1 : 0] wr_bin_ptr;
    reg [PTR_WIDTH-1 : 0] wr_gray_ptr_reg;
    reg wr_full_reg;
    reg [PTR_WIDTH-1 : 0] rd_gray_sync_reg;
    reg wr_overflow_reg;

    wire wr_full_next;
    wire [PTR_WIDTH-1 : 0] wr_bin_next;
    wire [PTR_WIDTH-1 : 0] wr_gray_next;

    always @(posedge wr_clk or negedge wr_rst_n)
    begin
        if(!wr_rst_n)
        begin
            wr_bin_ptr          <= {PTR_WIDTH{1'b0}};
            wr_gray_ptr_reg     <= {PTR_WIDTH{1'b0}};
            wr_full_reg         <= 1'b0;
            wr_overflow_reg     <= 1'b0;
            rd_gray_sync_reg    <= {PTR_WIDTH{1'b0}};
        end
        else
        begin
            // Capture synchronized read pointer
            rd_gray_sync_reg <= rd_gray_sync;
        end
    end

```

```

// Update write pointer
wr_bin_ptr      <= wr_bin_next;
wr_gray_ptr_reg <= wr_gray_next;

// Update full flag
wr_full_reg <= wr_full_next;

// Overflow detection (write when full)
if (wr_en && wr_full_reg)
    wr_overflow_reg <= 1'b1;
end

end

assign wr_allow      = (wr_en && !wr_full);
assign wr_addr       = wr_bin_ptr[ADDR_WIDTH-1:0];
assign wr_bin_next   = wr_bin_ptr + (wr_allow ? 1'b1 : 1'b0);
assign wr_gray_next  = wr_bin_next ^ (wr_bin_next >> 1);
assign wr_full_next  =
    (wr_gray_next ==
        {~rd_gray_sync_reg[PTR_WIDTH-1:PTR_WIDTH-2],
         // inverted top 2 bits of rd_gray_sync_reg,
         rd_gray_sync_reg[PTR_WIDTH-3:0]}) ;
    // remaining lower bits unchanged

assign wr_gray_ptr = wr_gray_ptr_reg;
assign wr_full     = wr_full_reg;
assign wr_overflow = wr_overflow_reg;

```

```
endmodule
```

```
//READ DOMAIN
```

```
module rd_domain #(
parameter DATA_WIDTH = 32,
          DEPTH = 16,
          ADDR_WIDTH = $clog2(DEPTH),
          PTR_WIDTH = ADDR_WIDTH + 1,
          SYNC_STAGES = 2 )(

input rd_clk,
input rd_rst_n,
input rd_en,
input [PTR_WIDTH-1 : 0]wr_gray_sync,

output [PTR_WIDTH-1 : 0]rd_gray_ptr,
output [ADDR_WIDTH-1 : 0]rd_addr,
output rd_empty,
output rd_valid,
output rd_underflow

);

reg [PTR_WIDTH-1 : 0]rd_bin_ptr;
reg [PTR_WIDTH-1 : 0]rd_gray_ptr_reg;
```

```

reg rd_empty_reg;
reg rd_underflow_reg;
reg [PTR_WIDTH-1 : 0]wr_gray_sync_reg;

wire [PTR_WIDTH-1 : 0]rd_bin_next;
wire [PTR_WIDTH-1 : 0]rd_gray_next;
wire rd_empty_next;
wire rd_allow;

always @(posedge rd_clk or negedge rd_rst_n)
begin
    if(!rd_rst_n)
    begin
        rd_bin_ptr          <= {PTR_WIDTH{1'b0}};
        rd_gray_ptr_reg     <= {PTR_WIDTH{1'b0}};
        rd_empty_reg        <= 1'b1;
        rd_underflow_reg    <= 1'b0;
        wr_gray_sync_reg    <= {PTR_WIDTH{1'b0}};
    end
    else
    begin

        wr_gray_sync_reg    <= wr_gray_sync;

        rd_bin_ptr          <= rd_bin_next;
        rd_gray_ptr_reg     <= rd_gray_next;
    end
end

```

```

    rd_empty_reg <= rd_empty_next;

    if(rd_en && rd_empty_reg)
        rd_underflow_reg <= 1'b1;
    end
end

assign rd_allow      = rd_en && !rd_empty_reg;
assign rd_bin_next   = rd_bin_ptr + (rd_allow ? 1 : 0);
assign rd_gray_next  = rd_bin_next ^ (rd_bin_next >> 1);
assign rd_empty_next = rd_gray_next == wr_gray_sync_reg;

assign rd_gray_ptr   = rd_gray_ptr_reg;
assign rd_addr       = rd_bin_ptr[ADDR_WIDTH-1 : 0];
assign rd_empty      = rd_empty_reg;
assign rd_valid      = rd_allow;
assign rd_underflow  = rd_underflow_reg;

endmodule

// POINTER SYNCHRONIZER

module sync_ptr #(
parameter PTR_WIDTH = 5,

```

```

        SYNC_STAGES = 2 )(

    input clk_dest,
    input rst_dest_n,
    input [PTR_WIDTH-1 : 0]ptr_gray_src,
                                // Gray pointer from source domain
    output [PTR_WIDTH-1 : 0]ptr_gray_sync
                                // Safe synchronized pointer
);

reg [PTR_WIDTH-1 : 0] sync_reg [0 : SYNC_STAGES-1];
integer i;

always @(posedge clk_dest or negedge rst_dest_n)
begin
    if(!rst_dest_n)
    begin
        for( i = 0; i < (SYNC_STAGES); i=i+1)
            sync_reg[i] <= {PTR_WIDTH{1'b0}};
        end
    else
    begin
        sync_reg[0] <= ptr_gray_src;

        for(i = 1; i < SYNC_STAGES; i = i + 1)
            sync_reg[i] <= sync_reg[i-1];
        end
    end
end

```

```

end

assign ptr_gray_sync = sync_reg[SYNC_STAGES-1];
endmodule


// FIFO MEMORY UNIT

module fifo_mem #(
parameter DATA_WIDTH = 32,
          DEPTH = 16,
          ADDR_WIDTH = $clog2(DEPTH)
        )(
input wr_clk,
input [DATA_WIDTH-1 : 0] wr_data,
input wr_en,
input [ADDR_WIDTH-1 : 0] wr_addr,

input rd_clk,
input [ADDR_WIDTH-1 : 0] rd_addr,
output reg [DATA_WIDTH-1 : 0] rd_data
);

reg [DATA_WIDTH-1 : 0] mem [0 : DEPTH-1];

always @(posedge wr_clk)

```

```

        if(wr_en)
            mem[wr_addr] <= wr_data;

always @(posedge rd_clk)
    rd_data <= mem[rd_addr];
endmodule


// DIAGNOSTIC UNIT

module fifo_diag#(
    parameter PTR_WIDTH = 5,
        STUCK_THRESHOLD = 16
    ) (
    // Write domain signals
    input wr_clk,
    input wr_rst_n,
    input wr_en,
    input wr_full,
    input [PTR_WIDTH-1 : 0]wr_gray_ptr,
    input wr_overflow,

    // Read domain signals
    input rd_clk,
    input rd_rst_n,
    input rd_en,
    input rd_empty,

```

```

    input [PTR_WIDTH-1 : 0]rd_gray_ptr,
    input rd_underflow,

    // Write domain outputs
    output wr_gray_error,
    output wr_overflow_sticky,
    output wr_pointer_stuck,

    // Read domain outputs
    output rd_gray_error,
    output rd_underflow_sticky,
    output rd_pointer_stuck
);

reg [PTR_WIDTH-1:0] prev_wr_gray;
reg wr_gray_error_reg;
wire [PTR_WIDTH-1:0] wr_diff;

reg [PTR_WIDTH-1:0] prev_rd_gray;
reg rd_gray_error_reg;
wire [PTR_WIDTH-1:0] rd_diff;

reg wr_overflow_sticky_reg;
reg rd_underflow_sticky_reg;

reg [$clog2(STUCK_THRESHOLD+1)-1:0] wr_stuck_count;
reg wr_pointer_stuck_reg;

```

```

reg [$clog2(STUCK_THRESHOLD+1)-1:0] rd_stuck_count;
reg rd_pointer_stuck_reg;

always @(posedge wr_clk or negedge wr_rst_n)
begin
    if (!wr_rst_n) begin
        prev_wr_gray          <= {PTR_WIDTH{1'b0}};
        wr_gray_error_reg     <= 1'b0;
        wr_overflow_sticky_reg <= 1'b0;
        wr_stuck_count        <= 0;
        wr_pointer_stuck_reg  <= 1'b0;
    end
    else begin
        // Gray violation detection
        if (wr_diff != 0 && (wr_diff & (wr_diff - 1)) != 0)
            wr_gray_error_reg <= 1'b1;

        // Overflow sticky
        if (wr_overflow)
            wr_overflow_sticky_reg <= 1'b1;

        // WR Pointer Stuck
        if (wr_en && !wr_full)
            begin
                if (wr_diff == 0)
                    begin
                        if (wr_stuck_count == STUCK_THRESHOLD)

```

```

        wr_pointer_stuck_reg <= 1'b1;
    else
        wr_stuck_count <= wr_stuck_count + 1'b1;
    end

    else begin
        wr_stuck_count <= 0;
    end

end

else begin
    wr_stuck_count <= 0;
end

// Update previous pointer
prev_wr_gray <= wr_gray_ptr;

end

end

always @(posedge rd_clk or negedge rd_rst_n) begin
    if (!rd_rst_n) begin
        prev_rd_gray      <= {PTR_WIDTH{1'b0}};
        rd_gray_error_reg <= 1'b0;
        rd_underflow_sticky_reg <= 1'b0;
        rd_stuck_count      <= 0;
        rd_pointer_stuck_reg <= 1'b0;
    end

    else begin

```

```

// Gray violation detection
if (rd_diff != 0 && (rd_diff & (rd_diff - 1)) != 0)
    rd_gray_error_reg <= 1'b1;

// Overflow sticky
if (rd_underflow)
    rd_underflow_sticky_reg <= 1'b1;

// RD Pointer Stuck
if (rd_en && !rd_empty)
begin
    if (rd_diff == 0)
    begin
        if (rd_stuck_count == STUCK_THRESHOLD)
            rd_pointer_stuck_reg <= 1'b1;
        else
            rd_stuck_count <= rd_stuck_count + 1'b1;
        end
    else begin
        rd_stuck_count <= 0;
    end
end
else begin
    rd_stuck_count <= 0;
end

// Update previous pointer

```

```

        prev_rd_gray <= rd_gray_ptr;
    end
end

assign wr_diff = wr_gray_ptr ^ prev_wr_gray;
assign rd_diff = rd_gray_ptr ^ prev_rd_gray;

assign wr_gray_error      = wr_gray_error_reg;
assign rd_gray_error      = rd_gray_error_reg;
assign wr_overflow_sticky = wr_overflow_sticky_reg;
assign rd_underflow_sticky = rd_underflow_sticky_reg;
assign wr_pointer_stuck   = wr_pointer_stuck_reg;
assign rd_pointer_stuck   = rd_pointer_stuck_reg;

endmodule

```

APPENDIX B: TESTBENCH

Note: The following testbench is used to verify the functionality of the asynchronous FIFO under independent clock domains. It includes scenarios for normal operation, overflow, and underflow conditions.

```

// ASYNCHRONOUS FIFO BASIC TESTBENCH
module tb_async_fifo_basic;

parameter DATA_WIDTH = 32;
parameter DEPTH       = 16;

```

```

reg wr_clk;
reg rd_clk;
reg wr_rst_n;
reg rd_rst_n;

reg wr_en;
reg [DATA_WIDTH-1:0] wr_data;
wire wr_full;
wire wr_overflow;

reg rd_en;
wire [DATA_WIDTH-1:0] rd_data;
wire rd_empty;
wire rd_valid;
wire rd_underflow;

// Diagnostic outputs
wire wr_gray_error;
wire wr_overflow_sticky;
wire wr_pointer_stuck;
wire rd_gray_error;
wire rd_underflow_sticky;
wire rd_pointer_stuck;

// DUT
async_fifo_top #(

```

```

        .DATA_WIDTH(DATA_WIDTH),
        .DEPTH(DEPTH)
    ) DUT (

        .wr_clk(wr_clk),
        .wr_rst_n(wr_rst_n),
        .wr_en(wr_en),
        .wr_data(wr_data),
        .wr_full(wr_full),
        .wr_overflow(wr_overflow),

        .rd_clk(rd_clk),
        .rd_rst_n(rd_rst_n),
        .rd_en(rd_en),
        .rd_data(rd_data),
        .rd_empty(rd_empty),
        .rd_valid(rd_valid),
        .rd_underflow(rd_underflow),

        .wr_gray_error(wr_gray_error),
        .wr_overflow_sticky(wr_overflow_sticky),
        .wr_pointer_stuck(wr_pointer_stuck),

        .rd_gray_error(rd_gray_error),
        .rd_underflow_sticky(rd_underflow_sticky),
        .rd_pointer_stuck(rd_pointer_stuck)
    );

```

```

// Clock Generation (Asynchronous)

initial wr_clk = 0;
always #5 wr_clk = ~wr_clk;    // 10ns period

initial rd_clk = 0;
always #7 rd_clk = ~rd_clk;    // 14ns period

// Test Sequence
integer i;

initial begin
    $display("Starting Basic FIFO Test");

    wr_rst_n = 0;
    rd_rst_n = 0;
    wr_en     = 0;
    rd_en     = 0;
    wr_data   = 0;

    #30;
    wr_rst_n = 1;
    rd_rst_n = 1;

    // Write Phase

```

```

$display("Writing data...");
for (i = 0; i < DEPTH; i = i + 1) begin
    @(posedge wr_clk);
    wr_en    = 1;
    wr_data = i;
end

@(posedge wr_clk);
wr_en = 0;

// Try Overflow
@(posedge wr_clk);
wr_en = 1;
wr_data = 32'hDEADBEEF;

@(posedge wr_clk);
wr_en = 0;

// Read Phase
$display("Reading data...");

rd_en = 1;

repeat (DEPTH) begin
    @(posedge rd_clk);
    if (rd_valid)
        $display("Read Data = %0d", rd_data);
end

```

```

end

rd_en = 0;

// Try Underflow
@(posedge rd_clk);
rd_en = 1;

@(posedge rd_clk);
rd_en = 0;

// Print Diagnostic Status
#50;
$display("Diagnostics:");
$display("wr_overflow_sticky = %0d", wr_overflow_sticky);
$display("rd_underflow_sticky = %0d", rd_underflow_sticky);
$display("wr_gray_error = %0d", wr_gray_error);
$display("rd_gray_error = %0d", rd_gray_error);
$display("wr_pointer_stuck = %0d", wr_pointer_stuck);
$display("rd_pointer_stuck = %0d", rd_pointer_stuck);

#100;
$finish;

end

endmodule

```

```

//      ASYNCHRONOUS FIFO BASIC TESTBENCH

module fault_inject_tb;

parameter DATA_WIDTH = 32;
parameter DEPTH      = 16;

reg wr_clk;
reg rd_clk;
reg wr_rst_n;
reg rd_rst_n;

reg wr_en;
reg [DATA_WIDTH-1:0] wr_data;
wire wr_full;
wire wr_overflow;

reg rd_en;
wire [DATA_WIDTH-1:0] rd_data;
wire rd_empty;
wire rd_valid;
wire rd_underflow;

// Diagnostic outputs
wire wr_gray_error;
wire wr_overflow_sticky;
wire wr_pointer_stuck;
wire rd_gray_error;

```

```

wire rd_underflow_sticky;
wire rd_pointer_stuck;

// DUT
async_fifo_top #(
    .DATA_WIDTH(DATA_WIDTH),
    .DEPTH(DEPTH)
) DUT (

    .wr_clk(wr_clk),
    .wr_rst_n(wr_rst_n),
    .wr_en(wr_en),
    .wr_data(wr_data),
    .wr_full(wr_full),
    .wr_overflow(wr_overflow),

    .rd_clk(rd_clk),
    .rd_rst_n(rd_rst_n),
    .rd_en(rd_en),
    .rd_data(rd_data),
    .rd_empty(rd_empty),
    .rd_valid(rd_valid),
    .rd_underflow(rd_underflow),

    .wr_gray_error(wr_gray_error),
    .wr_overflow_sticky(wr_overflow_sticky),
    .wr_pointer_stuck(wr_pointer_stuck),

```

```

        .rd_gray_error(rd_gray_error),
        .rd_underflow_sticky(rd_underflow_sticky),
        .rd_pointer_stuck(rd_pointer_stuck)
    );

// Clock Generation
initial wr_clk = 0;
always #5 wr_clk = ~wr_clk;    // 10ns period
initial rd_clk = 0;
always #7 rd_clk = ~rd_clk;    // 14ns period

// Test Sequence
integer i;
initial begin
    $display("Starting FIFO Fault Injection Test");
    wr_rst_n = 0;
    rd_rst_n = 0;
    wr_en     = 0;
    rd_en     = 0;
    wr_data   = 0;
    #30;
    wr_rst_n = 1;
    rd_rst_n = 1;

    // Normal Write Phase
    $display("Writing data...");

```

```

for (i = 0; i < DEPTH; i = i + 1) begin
    @(posedge wr_clk);
    wr_en    = 1;
    wr_data = i;
end
@(posedge wr_clk);
wr_en = 0;

// Overflow Condition
$display("Triggering Overflow...");
@(posedge wr_clk);
wr_en    = 1;
wr_data = 32'hDEADBEEF;
@(posedge wr_clk);
wr_en = 0;

// Normal Read Phase
$display("Reading data...");
rd_en = 1;
repeat (DEPTH) begin
    @(posedge rd_clk);
    if (rd_valid)
        $display("Read Data = %0d", rd_data);
end
rd_en = 0;

// Underflow Condition

```

```

$display("Triggering Underflow...");
@(posedge rd_clk);
rd_en = 1;
@(posedge rd_clk);
rd_en = 0;

// Gray Code Fault Injection
$display("Injecting Gray Code Fault...");
#50;
force DUT.u_fifo_diag.prev_wr_gray = 5'b00000;
force DUT.wr_gray_ptr              = 5'b11111;
#20;
release DUT.u_fifo_diag.prev_wr_gray;
release DUT.wr_gray_ptr;

// Pointer Stuck Fault Injection
$display("Injecting Pointer Stuck Fault...");
// Make sure FIFO is NOT full
rd_en = 1;
repeat(4)
    @(posedge rd_clk);
rd_en = 0;
// Enable writes again
wr_en = 1;
wr_data = 32'hAAAA5555;
// Freeze pointer
force DUT.wr_gray_ptr = DUT.wr_gray_ptr;

```

```

// Wait long enough to exceed threshold
#400;

// Release pointer
release DUT.wr_gray_ptr;

wr_en = 0;

// Final Diagnostic Display
#100;
$display("=====");
$display("Diagnostic Results");
$display("=====");

$display("wr_overflow_sticky = %0d", wr_overflow_sticky);
$display("rd_underflow_sticky = %0d", rd_underflow_sticky);

$display("wr_gray_error      = %0d", wr_gray_error);
$display("rd_gray_error      = %0d", rd_gray_error);

$display("wr_pointer_stuck    = %0d", wr_pointer_stuck);
$display("rd_pointer_stuck    = %0d", rd_pointer_stuck);
#100;
$finish;

end

endmodule

```